Syncopate Machine v1.0.0

Ruangyot Nanchiang
rayato159@gmail.com

Abstract

This private research note studies a small but annoying indie-game question: when is it worth replacing a readable enemy rule tree with a tiny learned policy? The testbed is a Godot/Rust 2D survival-game arena where the engine still owns collision, path execution, cooldowns, hit validation, and animation. The learned controller only chooses tactical actions for a dynamic zombie group. The latest model, *Syncopate Machine v1.0.0*, is a tiny grouped-attention entity decoder with RoPE, shared key/value heads, a D-Flash-style streamed attention recurrence, SIMD dot products in the Rust runtime, and per-entity Q-heads. The current version removes stand-still attack actions from both sides: zombies select only positioning or dash actions, then the engine attacks automatically when range, facing, cooldown, and dot-product checks pass; the player bot uses `ATTACK_WHILE_MOVE` combo actions that move, face, and then strike. The rule expert is also made less trivial: close zombies may dash in before the automatic attack, while zombies facing an attacking player may dash out. The fairest result is not a marketing win for AI: this stronger rule expert still defeats the armed player Q-bot slightly faster in held-out evaluation (1.160s mean TTK) than the selected learned model (1.194s), and it slightly wins the mixed checkpoint score. The useful conclusion is practical: tiny AI is worth exploring when behavior maintenance, state interactions, and action coverage matter more than a hand-tuned fastest kill path, but a compact expert remains brutally competitive in a small known arena.

1 Introduction

Small game AI usually starts with rules because rules ship. A designer writes `if close then attack`, adds a cooldown, adds a side-step, adds a player stamina check, and the enemy suddenly feels alive enough. That path is cheap, debuggable, and sane for an indie project.

The failure mode is scale. A rule tree that is readable at five cases becomes ugly at fifty cases: player stamina, player facing, attack cooldown, local obstacles, zombie spacing, swarm count, front pressure, side pressure, back pressure, dodge windows, and recovery punish timing. None of those conditions is hard alone. The mess appears when they interact.

This note asks a deliberately narrow question:

> *Can a tiny learned tactical policy replace part of the rule layer in a small action game without taking over the whole game simulation?*

The answer is currently mixed. Learned policies can become dangerous and can spread action usage across tactical roles, but a compact handcrafted expert can still win on raw time-to-kill. That is not a failure; it is the actual tradeoff a solo developer needs to know. If the target behavior is a direct fastest-kill controller, rules are brutally competitive. If the target behavior is broader tactical coverage under many state combinations, learned policies start to earn their keep.

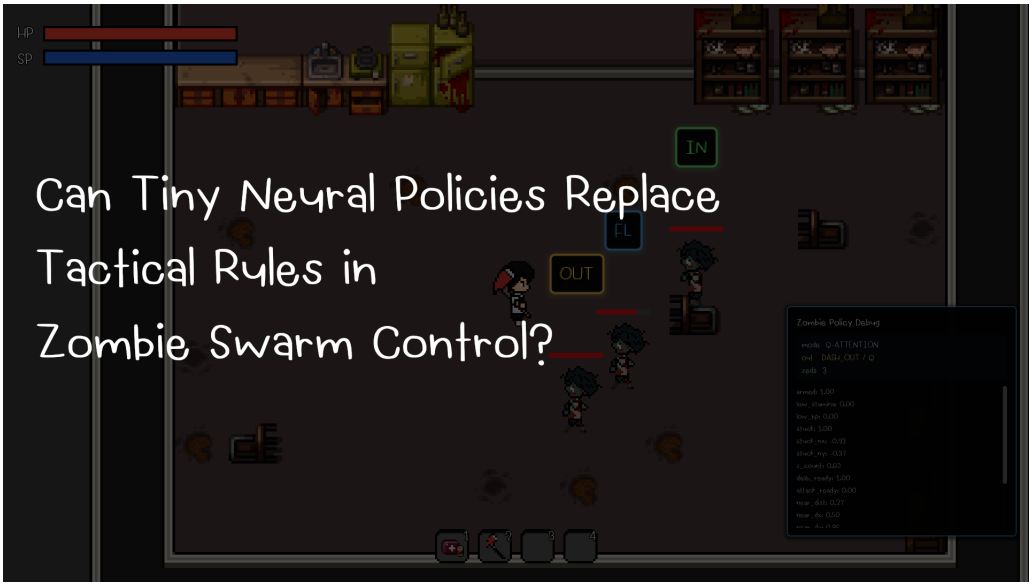


Figure 1: Debug view used for the current benchmark. The policy selects tactical actions, while the engine still executes movement, facing, hit validation, and cooldowns.

2 State and Actions Design

The policy does not control raw velocity. It reads a compact entity state and chooses high-level tactical actions. The game-side feasibility layer then turns those actions into path targets, facing, movement, attacks, and cooldown-safe execution. This separation keeps the model small and keeps Godot responsible for the systems that should remain deterministic.

The state is represented as one global player token and up to five zombie entity tokens:

$$s = \{g, e_1, \ldots, e_n\}, \qquad 1 \le n \le 5.$$

The global token has eight real-valued features:

$$g = [p_x, p_y, \eta_x, \eta_y, \text{stamina}, \text{attacking}, \text{hp}, n/5].$$

Each zombie token has eight real-valued features:

$$e_i = [z_x, z_y, \zeta_x, \zeta_y, \text{hp}_i, c_i^{atk}, c_i^{dash}, d_i].$$

Positions are normalized by arena scale, health and cooldowns are normalized by their natural maxima, and d_i is the normalized distance to the player.

The zombie model action set is deliberately attack-free: `PATH_FRONT`, `PATH_LEFT`, `PATH_RIGHT`, `PATH_BACK`, `DASH_UP`, `DASH_DOWN`, `DASH_LEFT`, and `DASH_RIGHT`. The path actions are not direct movement commands. They ask the engine to path toward a tactical slot around the player. For example, `PATH_BACK` asks a zombie to route toward the player's back side, but the engine still handles navigation, obstacle avoidance, collision, facing, and final attack execution. Zombie attack is an execution policy, not a learned action: if a zombie is close enough, attack cooldown is ready, its facing vector points through the player cone, and the dot-product threshold passes, the normal game attack state fires automatically.

The player bot is also a Q-policy during self-play. It uses a matching entity state and an action set containing rolls, movement, healing, pathing toward specific zombies, and two special combo actions: `ATTACK_WHILE_MOVE_1_COMBO` and `ATTACK_WHILE_MOVE_2_COMBOS`. These actions first move and face toward the target zombie, then strike only when the range and facing checks make the attack meaningful. The player is always armed in the latest self-play setup, so the zombie policy is trained against an aggressive duelist rather than a passive runner.

3 Model Design

3.1 Why not raw end-to-end control?

End-to-end control is the wrong hammer here. The goal is not to make the model learn Godot physics. The game already has pathfinding, collision, animation, combat arcs, stamina, hit validation, and cooldowns. Re-learning those pieces inside a tiny model would waste capacity and make debugging worse. The model is therefore only a tactical selector:

$$\pi_\theta(s) = \arg\max_{a \in \mathcal{A}} Q_\theta(s, a).$$

3.2 Entity decoder

The selected policy, *Syncopate Machine v1.0.0*, is a small grouped-attention entity decoder. It converts global and zombie features into tokens:

$$h_0 = E_g(g), \qquad h_i = E_z(e_i, r_i),$$

where r_i is a deterministic role/slot embedding. The model appends one action-query token per live zombie:

$$[h_0, h_1, \ldots, h_n, q_1, \ldots, q_n].$$

Each output query q_i predicts Q-values for zombie i.

For each layer, RMSNorm is applied before projection:

$$\hat{h}_t = \frac{h_t}{\sqrt{\frac{1}{d}\sum_{k=1}^{d} h_{t,k}^2 + \epsilon}} \odot w_{rms}.$$

Grouped attention uses more query heads than key/value heads:

$$Q_t^j = W_Q^j \hat{h}_t, \quad K_t^m = W_K^m \hat{h}_t, \quad V_t^m = W_V^m \hat{h}_t,$$

where query head j shares key/value head

$$m = \left\lfloor \frac{j H_{kv}}{H_q} \right\rfloor.$$

This is used because the policy needs multi-entity interaction, but the parameter budget is tiny. Sharing key/value heads gives the model attention-like geometry without paying full multi-head cost.

RoPE is applied to query and key channels so token order remains visible:

$$\mathrm{RoPE}(x_{2k}, x_{2k+1}, t) = (x_{2k}\cos\omega_{k,t} - x_{2k+1}\sin\omega_{k,t}, x_{2k}\sin\omega_{k,t} + x_{2k+1}\cos\omega_{k,t}).$$

The attention score is:

$$\alpha_{t,u}^j = \frac{\langle \mathrm{RoPE}(Q_t^j), \mathrm{RoPE}(K_u^m) \rangle}{\sqrt{d_h}} b(t, u).$$

Instead of a full softmax pass, the runtime uses a streamed D-Flash-style recurrence:

$$\lambda_u = \sigma(\alpha_u - \alpha_{u-1} + \log\lambda_{u-1}),$$

$$o_u = (1 - \lambda_u) o_{u-1} + \lambda_u V_u.$$

This is not a claim of GPU FlashAttention. It is a small CPU-friendly recurrence that keeps the runtime simple and avoids storing a full attention matrix.

Finally, the per-zombie output token becomes Q-values:

$$Q_\theta(e_i, a) = w_a^\top \mathrm{RMSNorm}(o_i) + b_a.$$

The current trainer learns the Q-heads with clipped temporal-difference updates over fixed deterministic embeddings and attention projections. That makes this closer to a tiny random-feature Q policy than a fully backpropagated Transformer. It is intentionally cheap and easy to embed in the Rust/Godot runtime.

3.3 Speculative action draft

The runtime includes a lightweight draft-and-verify path. A cheap role heuristic proposes a role path around the player. It never drafts a zombie attack, because attack is now handled by the game-side execution layer. The learned Q head then verifies the draft:

$$a^* = \arg\max_a Q(s, a),$$

If $Q(s, a_d) + \delta \geq Q(s, a^*)$, the runtime accepts $a = a_d$. Otherwise it rejects the draft and executes $a = a^*$. This borrows the engineering shape of speculative decoding–draft first, verify second–but it is not the exact probability-corrected language-model algorithm. Q-values are not token probabilities, so the correct adaptation is a margin-based verifier rather than distributional acceptance sampling.

4 Training Setup

Training is headless Rust self-play. No Godot window is opened. A random arena episode spawns an armed player bot and 1–5 zombies. Both sides collect transitions during the same episode and update after the episode ends. The latest default cadence is:

$$\Delta t = 0.10\,\mathrm{s}, \qquad T_{episode} = 60\,\mathrm{s}, \qquad N_{max} = 600.$$

The Godot demo defaults match this cadence: zombie tactical sampling uses 0.10s and tactical actions remain active for 0.20s.

Exploration uses epsilon decay:

$$\epsilon_k = \max(0.10, 1.0 \cdot 0.9975^k).$$

The temporal-difference target is:

$$y = r + \gamma \max_{a'} Q_\theta(s', a'), \qquad \gamma = 0.96.$$

The Q-head is updated with a clipped semi-gradient:

$$\Delta = \mathrm{clip}(Q_\theta(s, a) - y, -5, 5),$$

$$w_a \leftarrow w_a - \eta \Delta z, \qquad b_a \leftarrow b_a - \eta \Delta,$$

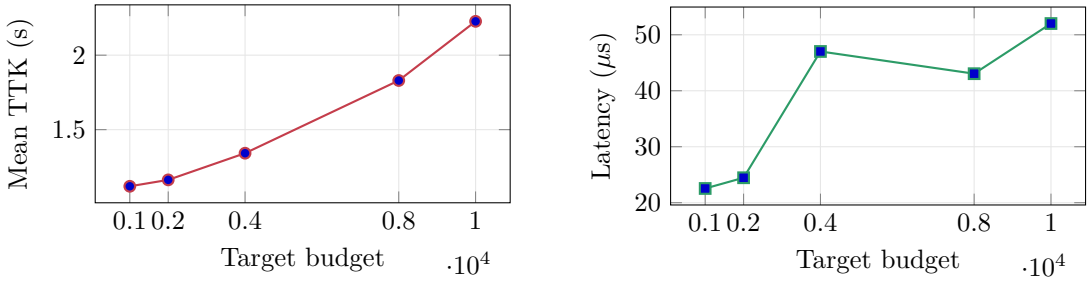
with $\eta = 0.004$. Checkpoints are evaluated every 500 episodes and the best score is retained. The score used for checkpoint selection is:

$$\mathrm{score} = 100 \cdot \mathrm{winrate} - \mathrm{TTK} + 8 \cdot H_{action} + 0.05 \cdot R_z.$$

This score deliberately mixes lethality and action coverage. It is not the same thing as pure time-to-kill, so both are reported.

Table 1: Capacity sweep at $\Delta t = 0.10$. Lower TTK is better; higher win-rate and score are better. Entropy is token-level action entropy in the Rust evaluator, not perceived animation variety in the Godot scene.

Budget	Params	Win	TTK (s)	Entropy	Score	Lat. (μs)
1k	968	1.000	1.120	0.623	105.506	22.544
2k	1,900	1.000	1.163	0.695	106.099	24.460
4k	3,968	1.000	1.342	0.295	102.764	47.020
8k	7,808	1.000	1.830	0.751	105.962	43.050
10k	10,208	1.000	2.227	0.732	105.372	52.011



Kill speed. Forward cost.

Figure 2: The 2k checkpoint is the selected indie-game point in this run: nearly as fast as the 1k model, broader in action usage, and still far below the 10k budget.

5 Results

5.1 Latest protocol change

The latest run changes the control contract rather than only changing weights. Zombie `ATTACK` is removed from the learned action set, and player attack actions are replaced by attack-while-move combo actions. This makes the comparison cleaner: both sides learn tactical movement choices, while the game engine owns the final attack feasibility checks. The reported results below are therefore not mixed with the older stand-still attack-action checkpoints. The rule expert is also upgraded for fairness: when a zombie is near the player it can randomly dash in before the automatic attack, and when the player is already attacking it can randomly dash out. The baseline is no longer just static path-slot rotation.

5.2 Capacity sweep

The 1k–10k sweep was rerun under the same 0.10s cadence after the attack-as-execution cleanup. The selected checkpoint is the 2k grouped-attention model. The 1k model kills slightly faster in this sweep, but the 2k model wins the mixed checkpoint score because it keeps stronger action coverage and player interaction while staying cheap.

5.3 Rule expert comparison

The final comparison uses the selected 2k attention checkpoint against the same armed player Q-policy. The upgraded rule expert remains slightly faster on the primary TTK metric and also slightly wins the mixed score. The learned policy is close, but it does not beat the stronger handcrafted baseline in this arena.

This is the most important result in the note. The learned model is not useless: it defeats the player bot reliably and stays close to the expert. But the handcrafted expert is still the fastest

Table 2: Held-out comparison at $\Delta t = 0.10$, 256 episodes. Entropy is computed from high-level action tokens. A high value can come from rotating path slots even if the visible behavior still looks like repeated chase-and-attack.

Controller	Params	Win	TTK (s)	Entropy	Score
Rule expert	–	1.000	1.160	0.700	106.132
Syncopate Machine v1.0.0	1,900	1.000	1.194	0.686	106.002

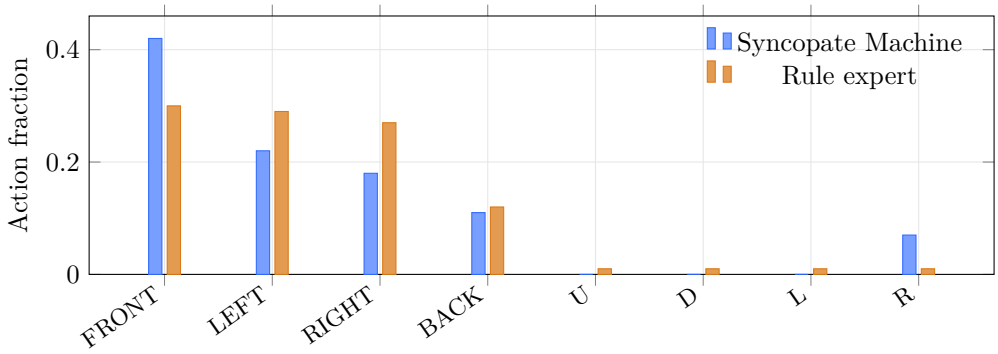


Figure 3: Action histogram after removing zombie `ATTACK` from the learned action space. The learned policy uses all path slots and occasionally `DASH_RIGHT`; the upgraded rule expert rotates path slots and uses a small amount of randomized dash-in/dash-out behavior. Higher diversity is not automatically better; here it only says the controller uses more high-level path tokens in the Rust evaluator.

killer because it is tuned directly for the arena's geometry and action rules. In a small arena with a known objective, a good rule can be brutally hard to beat.

6 Action Distribution

The histogram explains why the scalar entropy should not be over-read. A controller can be diverse and still worse at the primary objective. It can also look visually repetitive while receiving a nontrivial entropy score, because the metric counts high-level path-slot tokens rather than distinct animations. The rule expert's token entropy mainly comes from rotating the front, left, and right path-slot tokens, plus the new small probability of dash-in or dash-out. In the actual Godot scene those tokens all execute through the same readable A* chase/auto-attack pipeline. The selected learned policy uses `PATH_FRONT`, `PATH_LEFT`, `PATH_RIGHT`, `PATH_BACK`, and occasional `DASH_RIGHT`. That is closer to the desired action coverage than the older attack-action collapse. However, the rule expert still kills slightly faster because its slot rotation and dash reactions are hand-tuned to the arena's shortest pressure routes.

7 Discussion

For an indie game, the result is useful precisely because it is not clean. Rules are better when the target behavior is narrow, the map is known, and the designer can write the fastest tactic directly. Tiny learned policies become interesting when the designer wants a larger behavior surface: more state interactions, dynamic enemy counts, role-specific movement, and replayable experiments without writing another wall of if–else branches.

The 2k checkpoint is the current practical model. It is small enough to run in a Godot/Rust loop, uses grouped attention to look across entities, and costs only tens of microseconds in the Rust

sweep benchmark. The main cost is not runtime; it is training design. Reward shaping, evaluation seeds, action sanitization, and player-bot strength matter more than adding parameters.

The current implementation is also honest about its limits. The attention stack is fixed and the Q heads are trained with clipped TD updates. This keeps the model tiny and easy to export, but it also means the model is not a fully backpropagated Transformer policy. Future work should train the whole attention stack, improve dash rewards, and evaluate on more maps.

8 Conclusion

Syncopate Machine v1.0.0 says the quiet part clearly: small games should not use AI just because AI sounds cool. If a handcrafted controller is simple, inspectable, and already strong, keep it. In this benchmark the rule expert still wins on raw kill speed.

The learned policy is still worth keeping as a research tool. Syncopate Machine v1.0.0 is small, fast, almost as lethal as the expert, and better at exercising the front/side/back tactical interface. That makes it useful for exploring behavior coverage and for testing when rule authoring starts to become more expensive than training. The practical takeaway is not "attention beats rules." It is: use rules for known sharp tactics; use tiny learned policies when the tactical state space starts fighting back.

Code and Artifacts

The model-side Rust artifact is prepared as a separate public repository:

`Rayato159/tiny-zombie-q-policies`

The full Godot game source is intentionally separate.

References

[1] C. J. C. H. Watkins and P. Dayan. Q-learning. *Machine Learning*, 8:279–292, 1992.

[2] V. Mnih, K. Kavukcuoglu, D. Silver, et al. Human-level control through deep reinforcement learning. *Nature*, 518:529–533, 2015.

[3] A. Vaswani, N. Shazeer, N. Parmar, et al. Attention is all you need. In *NeurIPS*, 2017. arXiv:1706.03762v7, submitted 12 June 2017, revised 2 Aug. 2023. doi:10.48550/arXiv.1706.03762.

[4] J. Ainslie, J. Lee-Thorp, M. de Jong, Y. Zemlyanskiy, F. Lebron, and S. Sanghai. GQA: Training generalized multi-query transformer models from multi-head checkpoints. arXiv:2305.13245v3, submitted 22 May 2023, revised 23 Dec. 2023. doi:10.48550/arXiv.2305.13245.

[5] K. Alexandridis, V. Titopoulos, and G. Dimitrakopoulos. FLASH-D: FlashAttention with hidden softmax division. arXiv:2505.14201v1, submitted 20 May 2025. doi:10.48550/arXiv.2505.14201.

[6] The godot-rust project. The godot-rust book: User guide for gdext, the Rust binding for Godot 4. `https://godot-rust.github.io/book/`. Accessed 2026-06-13.